VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

DEEPU R (1BM25CS438)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Deepu R (1BM25CS438), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program-1a:

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

1)FCFS

2) SJF (pre-emptive & Non-preemptive)

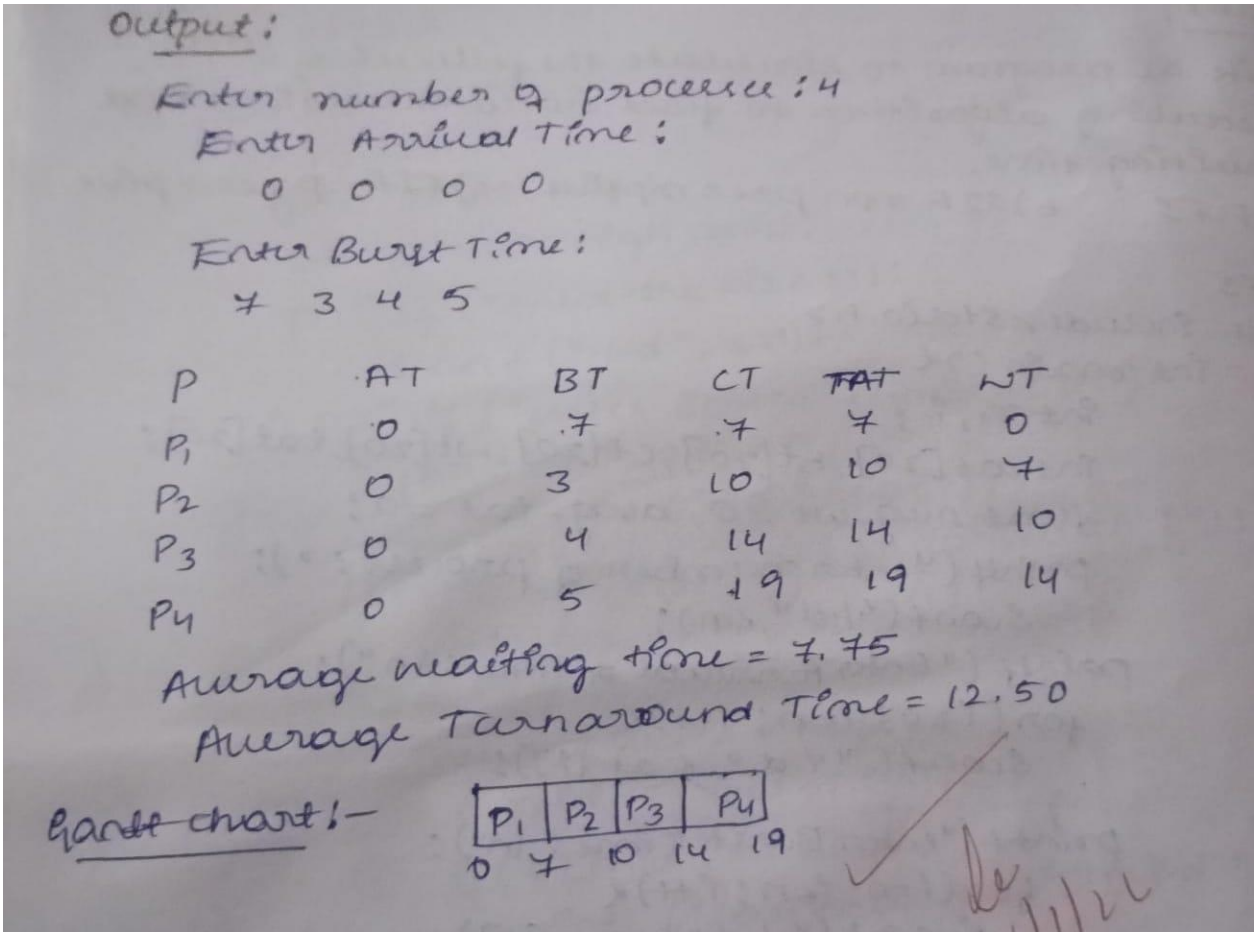
Code:

1)FCFS:

```
#include <stdio.h>
int main() {
   int n, i;
   int at[20], bt[20], ct[20], wt[20], tat[20];
   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++) {
      scanf("%d", &at[i]);
   }

   printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++) {
      scanf("%d", &bt[i]);
   }
   if(at[0] > 0)
      ct[0] = at[0] + bt[0];
   else
      ct[0] = bt[0];

   for(i = 1; i < n; i++) {
      if(ct[i-1] < at[i])
         ct[i] = at[i] + bt[i];
      else
```

```
            ct[i] = ct[i-1] + bt[i];
    }
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0;i < n;i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Waiting Time=%.2f\n",avg_wt/n);
    printf("Average Turnaround Time=%.2f\n",avg_tat/n);

    return 0;
}
```

OUTPUT:



2)SJF (pre-emptive)

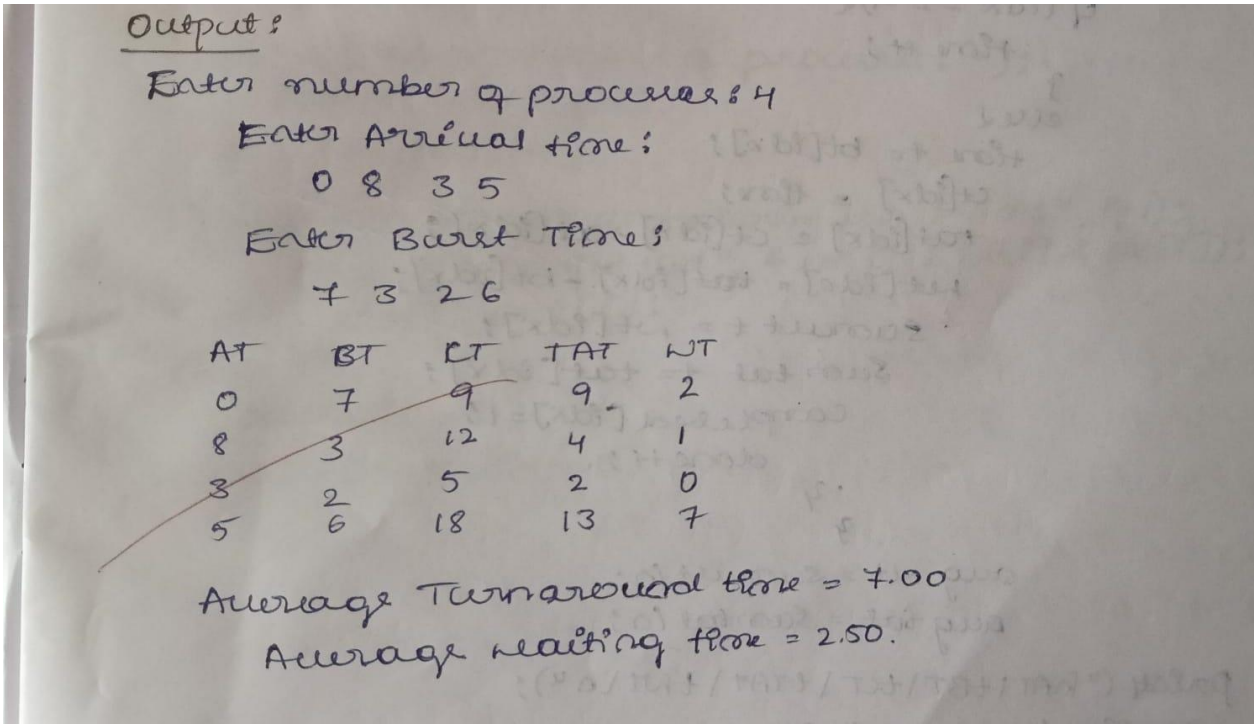
CODE:

```
#include <stdio.h>
int main() {
   int n, at[20], bt[20], rt[20], ct[20], tat[20], wt[20];
   int i, time = 0, completed = 0, shortest, min_rt;
   float avg_wt = 0, avg_tat = 0;
 printf("Enter number of processes: ");
   scanf("%d", &n);
 printf("Enter Arrival Time:\n");
   for(i = 0; i < n; i++) {
      scanf("%d", &at[i]);
   }
 printf("Enter Burst Time:\n");
   for(i = 0; i < n; i++) {
      scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
    }
while(completed < n) {
        shortest = -1;
        min_rt = 9999;

        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
                min_rt = rt[i];
                shortest = i;
            }
        }
 if(shortest == -1) {
            time++;
        }
        else {
            rt[shortest]--;
            time++;

            if(rt[shortest] == 0) {
                completed++;
              ct[shortest] = time;
                tat[shortest] = ct[shortest] - at[shortest];
                wt[shortest] = tat[shortest] - bt[shortest];

                if(wt[shortest] < 0)
                    wt[shortest] = 0;

                avg_tat += tat[shortest];
                avg_wt += wt[shortest];
            }
        }
    }
    printf("\nAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n", at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);
```

```
    printf("Average Waiting Time = %.2f\n", avg_wt/n);

    return 0;
}
```

OUTPUT:



SJF(Non-preemptive)

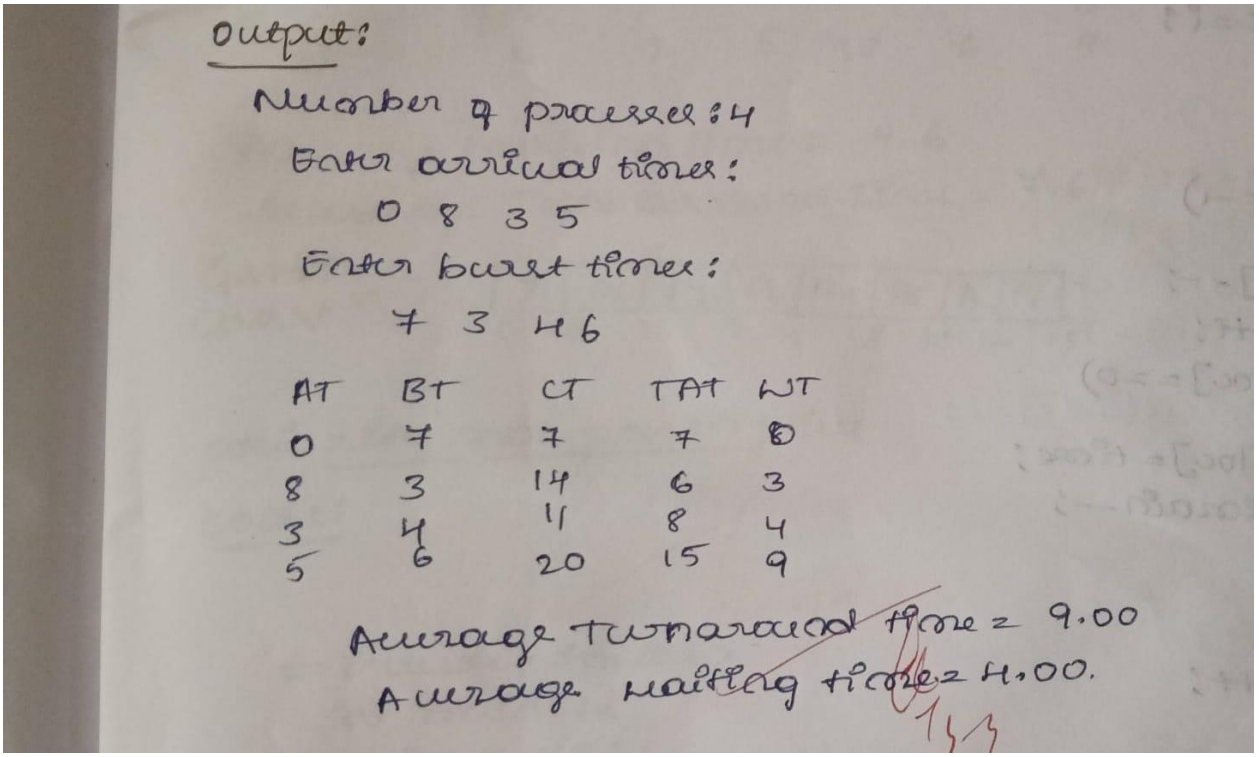
CODE:

```
#include<stdio.h>
int main() {
    int n, bt[20], at[20], ct[20], tat[20], wt[20], completed[20];
    float sumwt=0, sumtat=0, avgwt, avgtat;
    printf("Number of processes: ");
    scanf("%d", &n);
printf("Enter arrival times:\n");
    for(int i=0; i<n; i++) {
        scanf("%d", &at[i]);
    }
```

```
 printf("Enter burst times:\n");
   for(int i=0; i<n; i++) {
      scanf("%d", &bt[i]);
      completed[i] = 0;
   }
int time = 0, done = 0;
   while(done < n) {
      int idx = -1;
      int min_bt = 9999;
 for(int i=0; i<n; i++) {
         if(at[i] <= time && completed[i] == 0) {
            if(bt[i] < min_bt) {
               min_bt = bt[i];
               idx = i;
            }
         }
      }
 if(idx == -1) {
 time++;
      } else {
         time += bt[idx];
         ct[idx] = time;
         tat[idx] = ct[idx] - at[idx];
         wt[idx] = tat[idx] - bt[idx];
         sumwt += wt[idx];
         sumtat += tat[idx];
         completed[idx] = 1;
         done++;
      }
   }
  avgwt = sumwt / n;
```

```
    avgtat = sumtat / n;
 printf("\nAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n", at[i], bt[i], ct[i], tat[i], wt[i]);
    }
 printf("\nAverage Turnaround Time = %.2f\n", avgtat);
    printf("Average Waiting Time = %.2f\n", avgwt);
return 0;
}
```

OUTPUT:



Output:

Number of process: 4

Enter arrival times:

0 8 3 5

Enter burst times:

7 3 4 6

AT	BT	CT	TAT	WT
0	7	7	7	0
8	3	14	6	3
3	4	11	8	4
5	6	20	15	9

Average Turnaround time = 9.00

Average waiting time = 4.00.

PROGRAM-1b:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

1) Priority (pre-emptive & Non-pre-emptive)

2)Round Robin

1) **Priority (pre-emptive)**

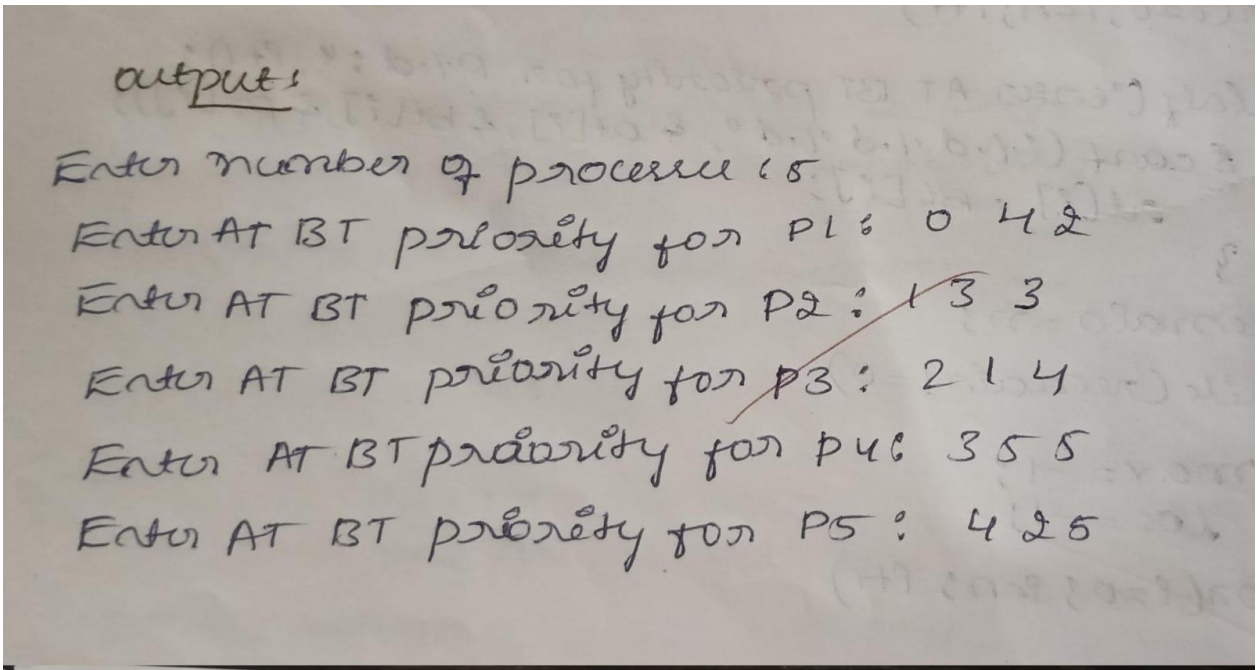
CODE:

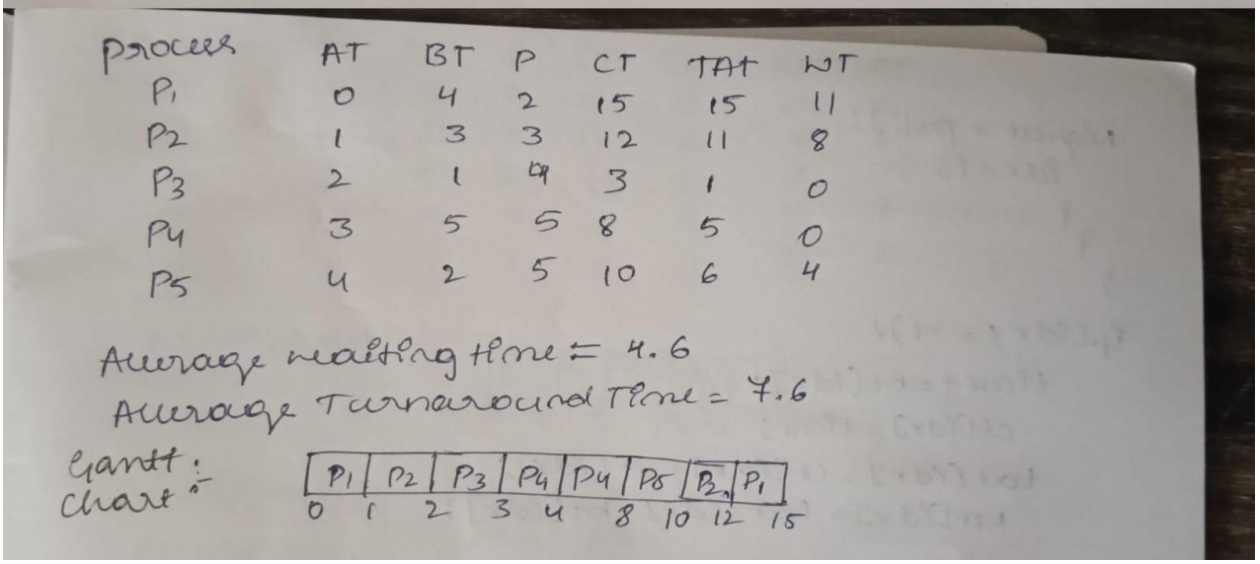
```
#include<stdio.h>
#include<limits.h>
struct process{
    int p_id;
    int at;
    int bt;
    int priority_no;
    int rem_time;
    int ct;
    int tat;
    int wt;
    int rt;
    int is_started;
};
int main(){
    int n;
    printf("enter the no of processes: ");
    scanf("%d",&n);
    struct process p[n];
    float total_wt=0,total_tat=0,total_rt=0;
    for(int i=0;i<n;i++){
        p[i].p_id=i+1;
        printf("\n process %d\n",p[i].p_id);
        printf("enter arrival time: ");
        scanf("%d",&p[i].at);
        printf("enter the burst time: ");
        scanf("%d",&p[i].bt);
        printf("enter the priority_no: ");
        scanf("%d",&p[i].priority_no);
        p[i].rem_time=p[i].bt;
        p[i].is_started=0;
    }
    int current_time=0;
    int completed_processes=0;
```

```
while(completed_processes<n){
    int highest_priority=INT_MAX;
    int shortest_process_index=-1;
    for(int i=0;i<n;i++){
        if(p[i].at<=current_time && p[i].rem_time>0){
            if(p[i].priority_no<highest_priority){
                highest_priority=p[i].priority_no;
                shortest_process_index=i;
            }
            else if(p[i].priority_no==highest_priority){
                if(p[i].at<p[shortest_process_index].at){
                    shortest_process_index=i;
                }
            }
        }
    }
    if(shortest_process_index!=-1){
        int i=shortest_process_index;
        if(p[i].is_started==0){
            p[i].rt=current_time-p[i].at;
            p[i].is_started=1;
        }
        p[i].rem_time--;
        if(p[i].rem_time==0){
            completed_processes++;
            p[i].ct=current_time+1;
            p[i].tat=p[i].ct-p[i].at;
            p[i].wt=p[i].tat-p[i].bt;
            total_tat+=p[i].tat;
            total_wt+=p[i].wt;
            total_rt+=p[i].rt;
        }
    }
    current_time++;
}
printf("pid\tarrival\tburst\tcompletion\tturnaround\twaiting\tresponse\n");
for(int i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\t\t%d\t\t%d\t%d\n",
          p[i].p_id,
          p[i].at,
          p[i].bt,
          p[i].priority_no,
          p[i].ct,
```

```
            p[i].tat,
            p[i].wt,
            p[i].rt);
    }
    printf("\n Avg turnaround time:%.2f",total_tat/n);
    printf("\n Avg waiting time:%.2f",total_wt/n);
    printf("\nAvg response time:%.2f",total_rt/n);
    return 0;
}
```

OUTPUT:





process	AT	BT	P	CT	TAT	WT
P1	0	4	2	15	15	11
P2	1	3	3	12	11	8
P3	2	1	4	3	1	0
P4	3	5	5	8	5	0
P5	4	2	5	10	6	4

Average waiting time = 4.6
Average Turnaround Time = 7.6

Gantt Chart:

Priority(Non-pre-emptive)
CODE:

```
#include <stdio.h>

int main() {
   int n = 5; // number of processes
   int at[5], bt[5], pr[5];
   int ct[5], tat[5], wt[5];
   int completed[5] = {0};

   int time = 0, done = 0;

   printf("Enter Arrival Time:\n");
   for(int i = 0; i < n; i++) {
      scanf("%d", &at[i]);
   }

   printf("Enter Burst Time:\n");
   for(int i = 0; i < n; i++) {
      scanf("%d", &bt[i]);
   }

   printf("Enter Priority (lower value = higher priority):\n");
   for(int i = 0; i < n; i++) {
      scanf("%d", &pr[i]);
   }

   while(done < n) {
      int idx = -1;
      int highest = 9999;
for(int i = 0; i < n; i++) {
         if(at[i] <= time && completed[i] == 0) {
            if(pr[i] < highest) {
               highest = pr[i];
               idx = i;
            }
         }
      }
```

```
        if(idx != -1) {
            time += bt[idx];
            ct[idx] = time;
            completed[idx] = 1;
            done++;
        } else {
            time++; // CPU idle
        }
    }
    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }
  float avg_tat = 0, avg_wt = 0;
    for(int i = 0; i < n; i++) {
        avg_tat += tat[i];
        avg_wt += wt[i];
    }
  printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", avg_wt/n);
  return 0;
}
```

OUTPUT:

output :

Enter number of process : 5

Enter Arrival time :

0 2 3 4 6

Enter Burst time :

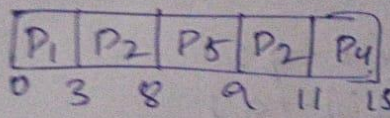
3 2 5 4 1

Enter the priority :

5 3 2 4 1

process	AT	BT	P	CT	TAT	WT	RT
P_1	0	3	5	3	3	0	0
P_2	2	2	3	11	9	7	7
P_3	3	5	2	8	5	0	0
P_4	4	4	4	15	11	7	7
P_5	6	1	1	9	3	2	2

Gantt chart :-



Average of TAT :- 6.2

Average of WT :- 16

2)Round Robin

CODE:

```
#include <stdio.h>
struct process {
   int pid;
   int at, bt, rt;
   int ct, tat, wt;
};
int main() {
   int n, tq;
   struct process p[10];
   int time = 0, remain, i;
 float total_tat = 0, total_wt = 0;
 printf("Enter number of processes: ");
   scanf("%d", &n); remain = n;
 for(i = 0; i < n; i++) {
       p[i].pid = i + 1;
       printf("Enter AT and BT for P%d: ", i+1);
       scanf("%d %d", &p[i].at, &p[i].bt);
       p[i].rt = p[i].bt;
   }
printf("Enter Time Quantum: ");
   scanf("%d", &tq);
   while(remain != 0) {
       int done = 1;
 for(i = 0; i < n; i++) {
 if(p[i].rt > 0 && p[i].at <= time) {
              done =  if(p[i].rt > tq) {
                 time += tq;
                 p[i].rt -= tq;
              } else {
                 time += p[i].rt;
                 p[i].rt = 0;
                 remain--;
                 p[i].ct = time;
                 p[i].tat = p[i].ct - p[i].at;
                 p[i].wt = p[i].tat - p[i].bt;
                 total_tat += p[i].tat;
                 total_wt += p[i].wt;
              }
```

```
            }
        }
    if(done)
            time++;
    }
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid, p[i].at, p[i].bt,
                p[i].ct, p[i].tat, p[i].wt);
    }
 printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
 return 0;
}
```

OUTPUT:

Enter number of process : 5

Enter Arrival Time :

0 5 1 6 7

Enter Burst time :

8 2 7 3 5

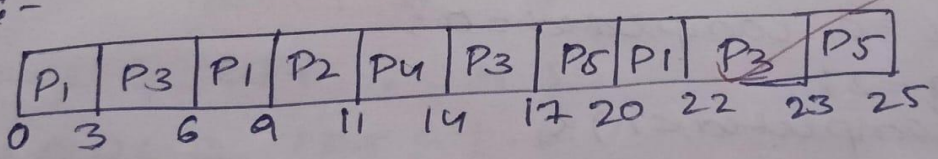
Enter Time Quantum : 3

Process	AT	BT	CT	TAT	WT
P_1	0	8	22	22	14
P_2	5	2	11	6	4
P_3	1	7	23	22	15
P_4	6	3	14	8	5
P_5	7	5	25	13	13

Average of TAT :- 14.2

Average of WT :- 10.2

gantt :-



PROGRAM-2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>
#define MAX 20

struct Process {
   int pid, at, bt, rt;
   int ct, tat, wt;
   int qno;
};

int main() {

   int n;
   struct Process p[MAX];

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter AT BT QueueNo(1-RR,2-FCFS):\n");

   for (int i = 0; i < n; i++) {
      p[i].pid = i + 1;
      scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].qno);
      p[i].rt = p[i].bt;
   }

   int time = 0, completed = 0;
   int tq = 2;

   while (completed < n) {

      int executed = 0;
```

```
for (int i = 0; i < n; i++) {

   if (p[i].qno == 1 &&
      p[i].rt > 0 &&
      p[i].at <= time) {

      executed = 1;

      int run = (p[i].rt > tq) ? tq : p[i].rt;

      time += run;
      p[i].rt -= run;

      if (p[i].rt == 0) {
         p[i].ct = time;
         completed++;
      }
   }
}

if (executed) continue;

int idx = -1;
int earliest = 99999;

for (int i = 0; i < n; i++) {
   if (p[i].qno == 2 &&
      p[i].rt > 0 &&
      p[i].at <= time) {

      if (p[i].at < earliest) {
         earliest = p[i].at;
         idx = i;
      }
   }
}

if (idx != -1) {
```

```
        executed = 1;

        while (p[idx].rt > 0) {
            int q1_ready = 0;

            for (int j = 0; j < n; j++) {
                if (p[j].qno == 1 &&
                    p[j].rt > 0 &&
                    p[j].at <= time) {
                    q1_ready = 1;
                    break;
                }
            }

            if (q1_ready)
                break;

            time++;
            p[idx].rt--;
        }

        if (p[idx].rt == 0) {
            p[idx].ct = time;
            completed++;
        }
    }

    if (!executed)
        time++;
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {

    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
```

```
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid,
               p[i].at,
               p[i].bt,
               p[i].ct,
               p[i].tat,
               p[i].wt);

        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
    }

    printf("\nAvg WT = %.2f", avg_wt / n);
    printf("\nAvg TAT = %.2f\n", avg_tat / n);

    return 0;
}
```

OUTPUT:

Output:

enter number of process : 4

enter AT no :

0 0 0 10

enter BT no :

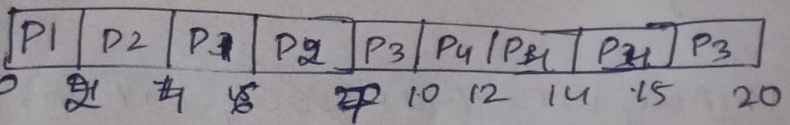
4 3 8 5

enter the queue no :

1 1 2 1

	Q	AT	BT	CT	TAT	WT
P_1	1	0	4	4	4	0
P_2	1	0	3	7	7	4
P_3	2	0	8	15	15	7
P_4	1	10	5	20	10	5

gantt chart :-



Average of wt :- 4.5

Average of TAT :- 9.6.

PROGRAM-3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

b) Earliest-deadline First

c) Proportional scheduling

a) Rate- Monotonic

CODE:

```
#include <stdio.h>
#include <math.h>

#define MAX 10

int find_lcm(int a, int b) {
   int max = (a > b) ? a : b;
   while (1) {
      if (max % a == 0 && max % b == 0)
         return max;
      max++;
   }
}

int main() {
   int n;
   printf("Enter the number of processes:");
   scanf("%d", &n);

   int C[MAX], P[MAX];

   printf("Enter the CPU burst times:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &C[i]);

   printf("Enter the time periods:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &P[i]);

   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {
```

```
        if (P[i] > P[j]) {
            int temp;

            temp = P[i]; P[i] = P[j]; P[j] = temp;
            temp = C[i]; C[i] = C[j]; C[j] = temp;
        }
    }
}

int lcm = P[0];
for (int i = 1; i < n; i++) {
    lcm = find_lcm(lcm, P[i]);
}

printf("LCM=%d\n\n", lcm);

printf("Rate Monotone Scheduling:\n");
printf("PID    Burst    Period\n");
for (int i = 0; i < n; i++) {
    printf("%d      %d        %d\n", i + 1, C[i], P[i]);
}

float U = 0;
for (int i = 0; i < n; i++) {
    U += (float)C[i] / P[i];
}

float bound = n * (pow(2, (float)1 / n) - 1);

printf("\n%f <= %f => %s\n", U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", lcm);

int remaining[MAX] = {0};
int current = -1;

for (int t = 0; t < lcm; t++) {

    for (int i = 0; i < n; i++) {
```

```
            if (t % P[i] == 0) {
                remaining[i] = C[i];
            }
        }

        int next = -1;
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                next = i;
                break;
            }
        }

        if (next != current) {
            if (next == -1)
                printf("%dms onwards: CPU is idle\n", t);
            else
                printf("%dms onwards: Process %d running\n", t, next + 1);
            current = next;
        }

        if (next != -1) {
            remaining[next]--;
        }
    }

    return 0;
}
```

OUTPUT:

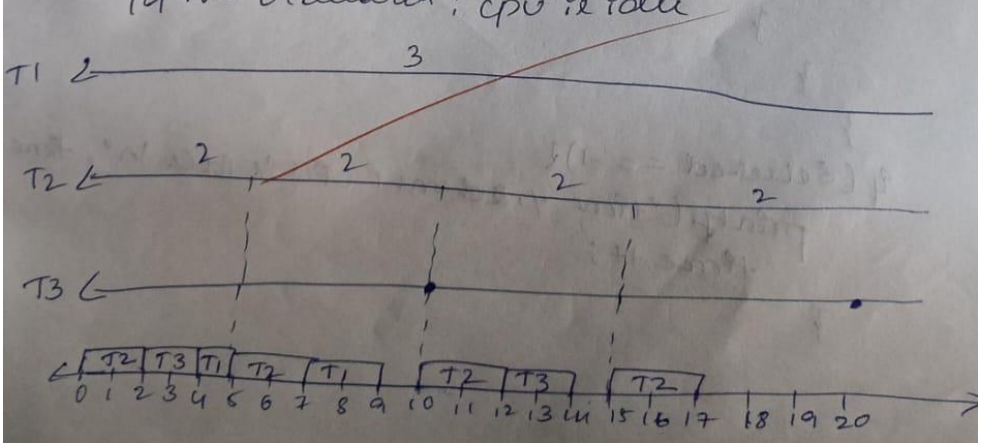
Enter the number of process : 2
Enter the CPU burst time:
3 2 2
enter the time periods:
20 5 10
LCM 20

Rate Monotone Scheduling:

PID	Burst	period
1	3	20
2	2	5
3	2	10

0.9000 <= 1 => true
Scheduling occurs for 20 ms.

0ms onwards : process 1 running
1ms onwards : process 1 running
2ms onwards : process 2 running
3ms onwards : process 2 running
4ms onwards : process 3 running
5ms onwards : process 1 running
6 ms onwards : process 1 running
7 ms onwards : process 3 running
8 ms onwards : process 3 running
9 ms onwards : CPU is idle
10 ms onwards : process 1 running
11 ms onwards : process 1 running
12 ms onwards : process 2 running
13 ms onwards : process 2 running
14ms onwards : CPU is idle
15 ms onwards : process 1 running
16 ms onwards : process 1 running
17ms onwards : CPU is idle
18 ms onwards : CPU is idle
19 ms onwards : CPU is idle



b) Earliest-deadline First

CODE:

```
#include <stdio.h>

#define MAX 10

int main() {
   int n;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int burst[MAX], deadline[MAX], period[MAX];
   int remaining[MAX], abs_deadline[MAX];

   printf("Enter Burst Time, Initial Deadline, Period:\n");
   for (int i = 0; i < n; i++) {
      scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
      remaining[i] = burst[i];
      abs_deadline[i] = deadline[i];
   }

   printf("\nPID\tBurst\tDeadline\tPeriod\n");
   for (int i = 0; i < n; i++) {
      printf("%d\t%d\t%d\t\t%d\n", i+1, burst[i], deadline[i], period[i]);
   }

   int time = 0, end_time;

   printf("\nEnter total scheduling time: ");
   scanf("%d", &end_time);

   printf("\nScheduling occurs for %d ms\n\n", end_time);

   while (time < end_time) {

      for (int i = 0; i < n; i++) {
         if (time != 0 && time % period[i] == 0) {
            remaining[i] = burst[i];
```

```
            abs_deadline[i] = time + deadline[i];
        }
    }

    int selected = -1;
    int min_deadline = 9999;

    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (abs_deadline[i] < min_deadline) {
                min_deadline = abs_deadline[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("%dms: CPU is idle.\n", time);
    } else {
        printf("%dms: Task %d is running.\n", time, selected + 1);
        remaining[selected]--; // execute
    }

    time++;
  }

  return 0;
}
```

OUTPUT:

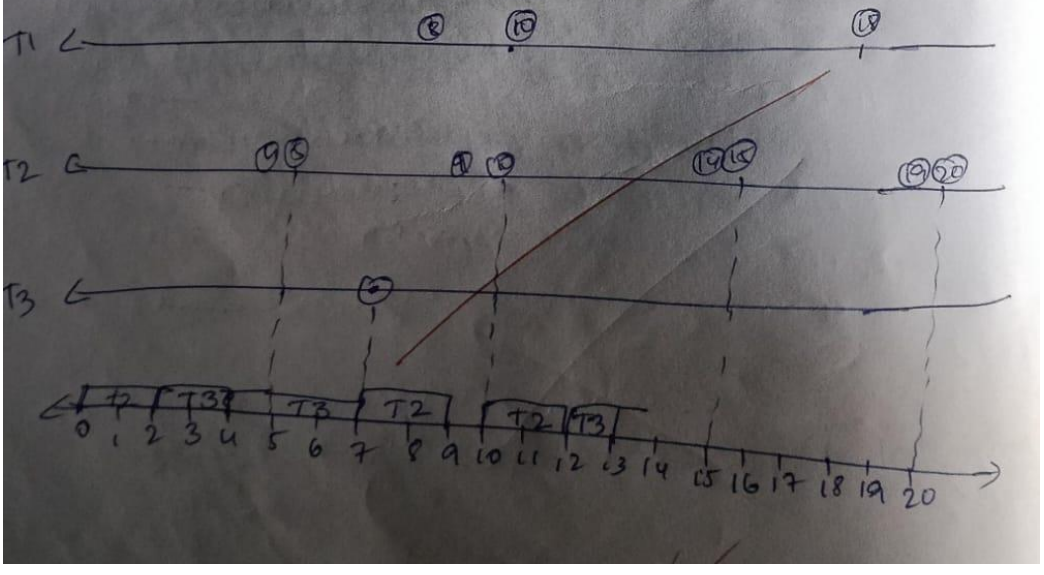
Output:

Enter number of task : 3
enter Burst time:
3 2 2
enter deadline:
7 4 8
enter periods:
20 5 10

PID	Burst	Deadline	period
1	3	7	20
2	2	4	5
3	2	8	10

Scheduling occurs one

One : task 2 is running
1 ne : task 2 is running
2 ne : task 1 is running
3 ne : task 1 is running
4 ne : task 2 is running
5 ne : Task 2 is running
6 ne : Task 3 is running.
7 ne : Task 3 is running
8 ne : Task 3 is running



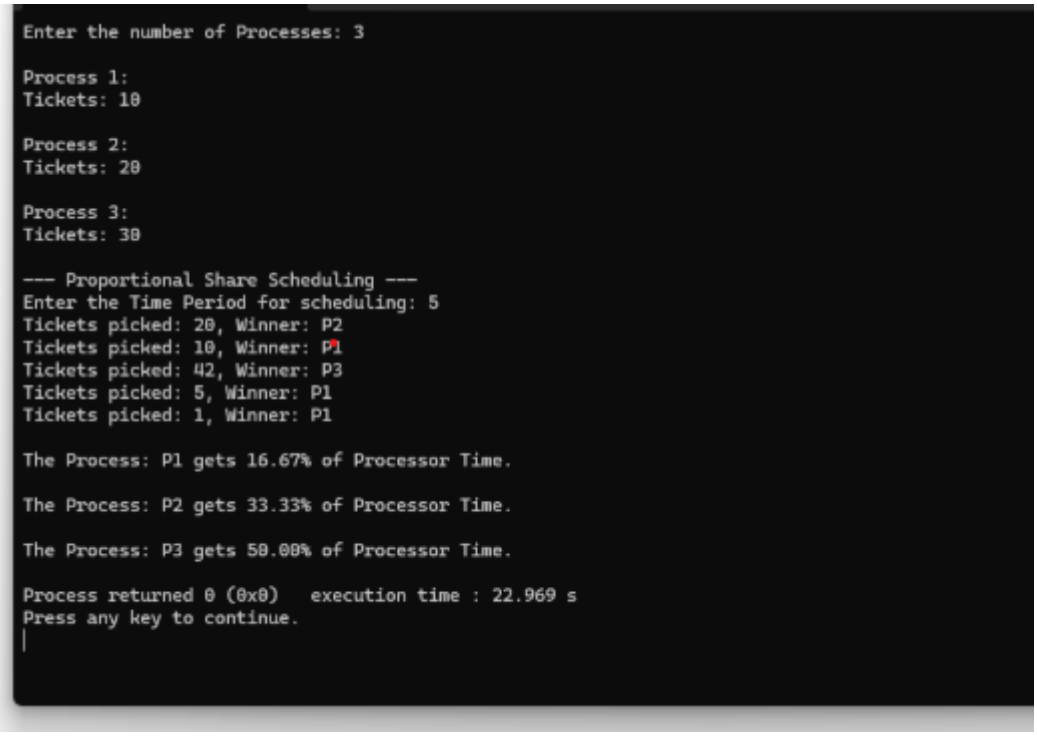
c) Proportional scheduling

CODE:

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
char name[5]; int tickets; }
Process;
int main() {
int n, total_tickets = 0; float total_T =0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
```

```
}
}
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));

}
return 0;
 }
```

OUTPUT:



PROGRAM-4:

Write a C program to simulate:

a) Producer-consumer problem using semaphores

b) Dining-Philosopher's problem.

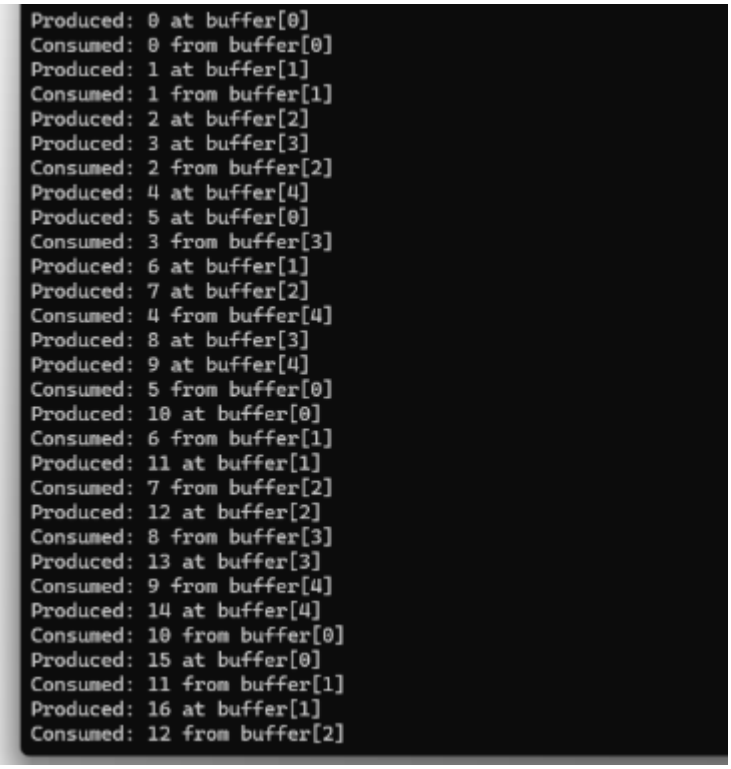
a)Producer-consumer problem using semaphores

CODE:

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty;
sem_t full;

void* producer(void* arg) {
int item, i = 0;
while (1) {
item = i++;
sem_wait(&empty);
pthread_mutex_lock(&mutex);
buffer[in] = item;
printf("Produced: %d at buffer[%d]\n", item, in);
in = (in + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex);
sem_post(&full);
sleep(1);
}
}
void* consumer(void* arg)

{
int item;
while (1)
{
sem_wait(&full);
pthread_mutex_lock(&mutex);
```

```
item = buffer[out];
printf("Consumed: %d from buffer[%d]\n", item, out);
out = (out + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex);
sem_post(&empty);
sleep(2);
}
}
int main()
{
pthread_t prod, cons;
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0); pthread_mutex_init(&mutex, NULL);
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);
pthread_join(prod, NULL);
pthread_join(cons, NULL);
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
return ;
}
```

OUTPUT:



.

b) Dining-Philosopher's problem.

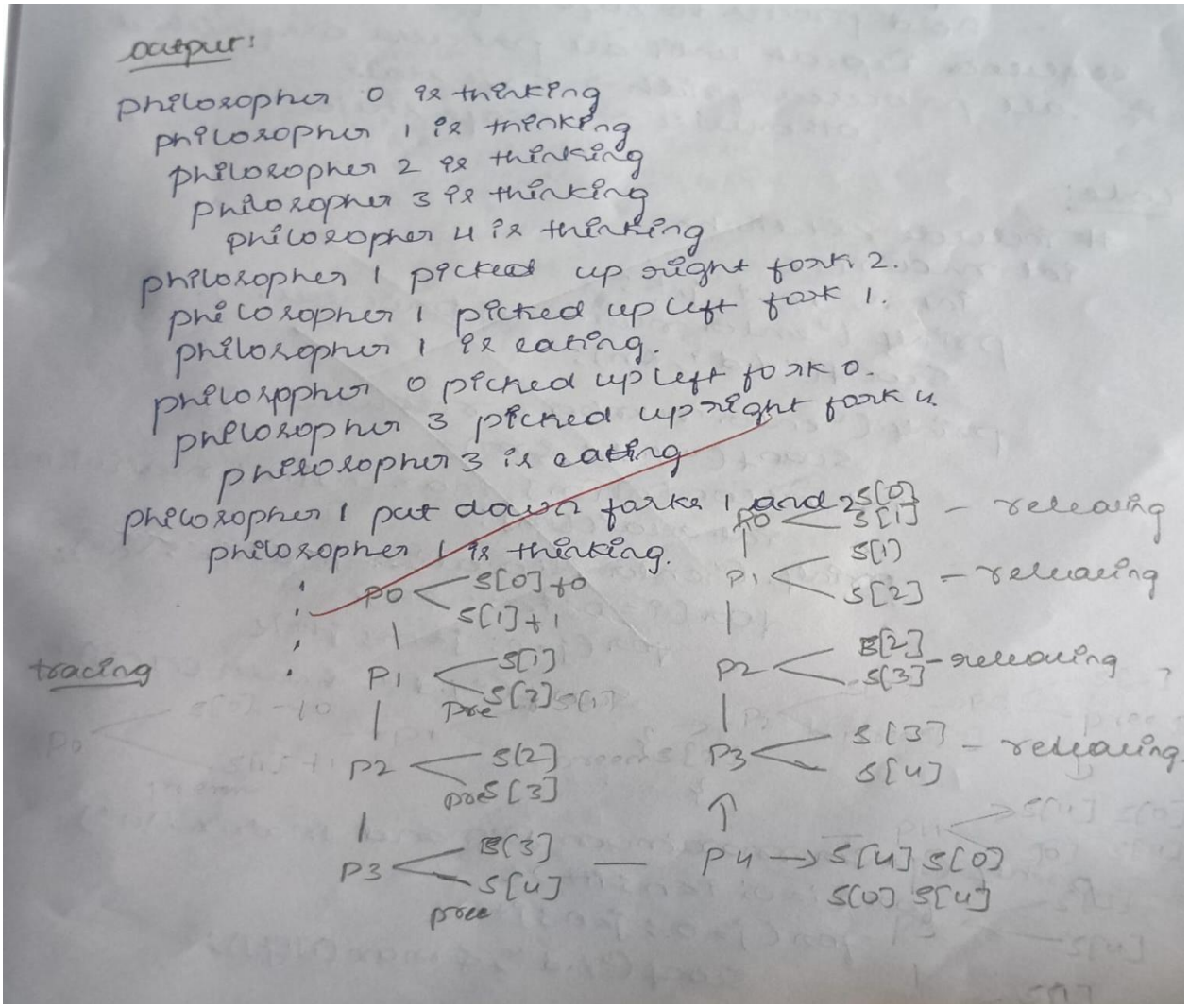
CODE:

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosophers[N];
void* philosopher(void* num)
{
int id = *(int*)num;
int left = id;
int right = (id + 1) % N;
while (1)
{
printf("Philosopher %d is thinking.\n", id);
sleep(1);
```

```
if (id % 2 == 0)
{

pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);

pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); }
else
{
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right);

printf("Philosopher %d picked up left fork %d.\n", id, left);
}

printf("Philosopher %d is eating.\n", id);
sleep(2);
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

int main() {
int i;
int ids[N];
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
for (i = 0; i < N; i++)

{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
}
for (i = 0; i < N; i++)
{
```

```
pthread_join(philosophers[i], NULL);
 }
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}
```

OUTPUT:



PROGRAM-5:

Write a C program to simulate

a) Banker's algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

a)Bankers Algorithm

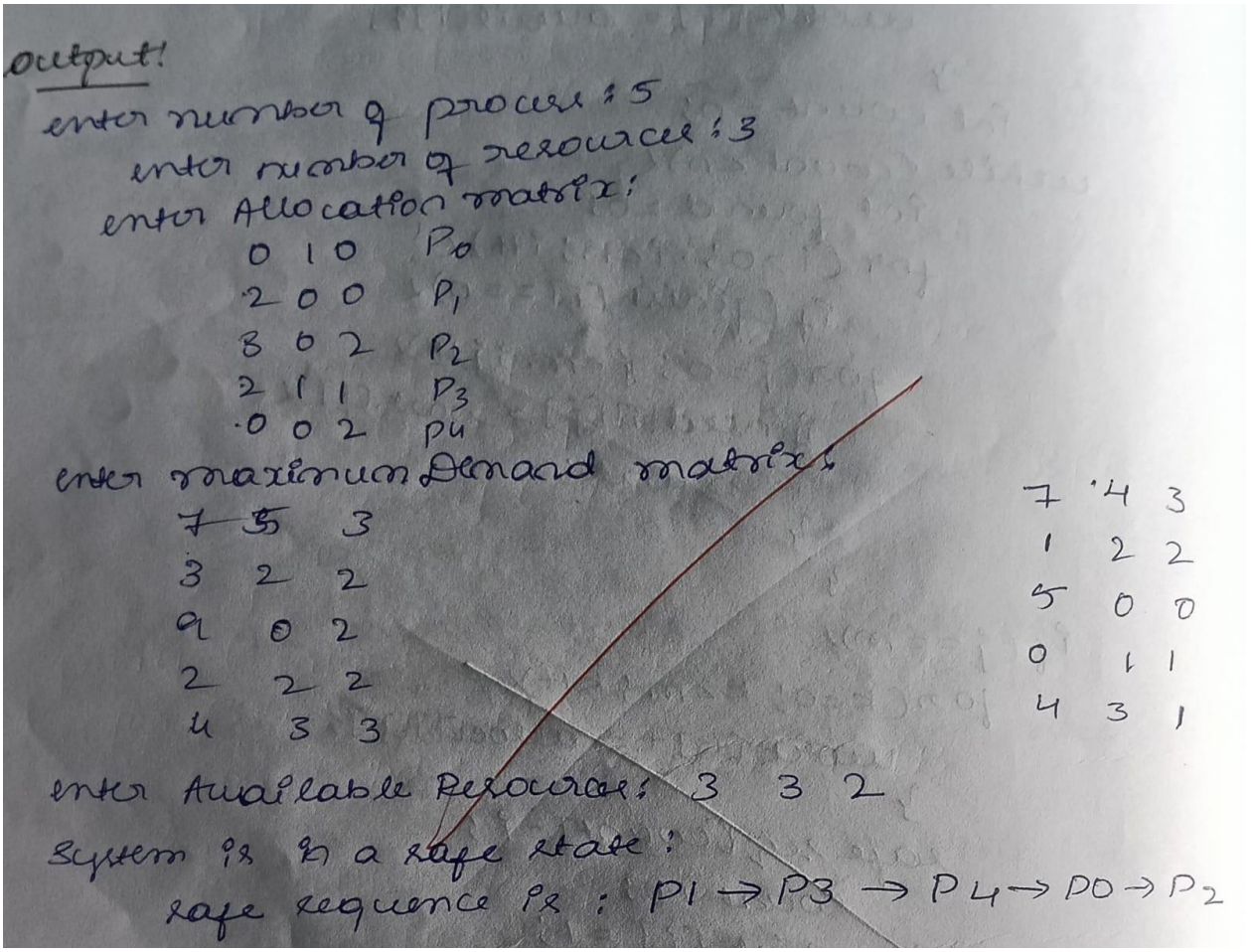
CODE:

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m];
   int avail[m];

   printf("\nEnter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("\nEnter Maximum Demand Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &max[i][j]);
      }
   }

   printf("\nEnter Available Resources:\n");
   for(i = 0; i < m; i++)
```

```
{
   scanf("%d", &avail[i]);
}
for(i = 0; i < n; i++)
{
   for(j = 0; j < m; j++)
   {
      need[i][j] = max[i][j] - alloc[i][j];
   }
}

int finish[n], safeSeq[n];
int work[m];

for(i = 0; i < n; i++)
   finish[i] = 0;

for(i = 0; i < m; i++)
   work[i] = avail[i];

int count = 0;

while(count < n)
{
   int found = 0;

   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         for(j = 0; j < m; j++)
         {
            if(need[i][j] > work[j])
               break;
         }

         if(j == m)
         {
            for(k = 0; k < m; k++)
               work[k] += alloc[i][k];

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
```

```
        }

        if(found == 0)
        {
            printf("\nSystem is NOT in a Safe State.\n");
            return 0;
        }
    }

    printf("\nSystem is in Safe State.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)
            printf(" -> ");
    }

    printf("\n");

    return 0;
}
```

OUTPUT:



b)Deadlock Algorithm

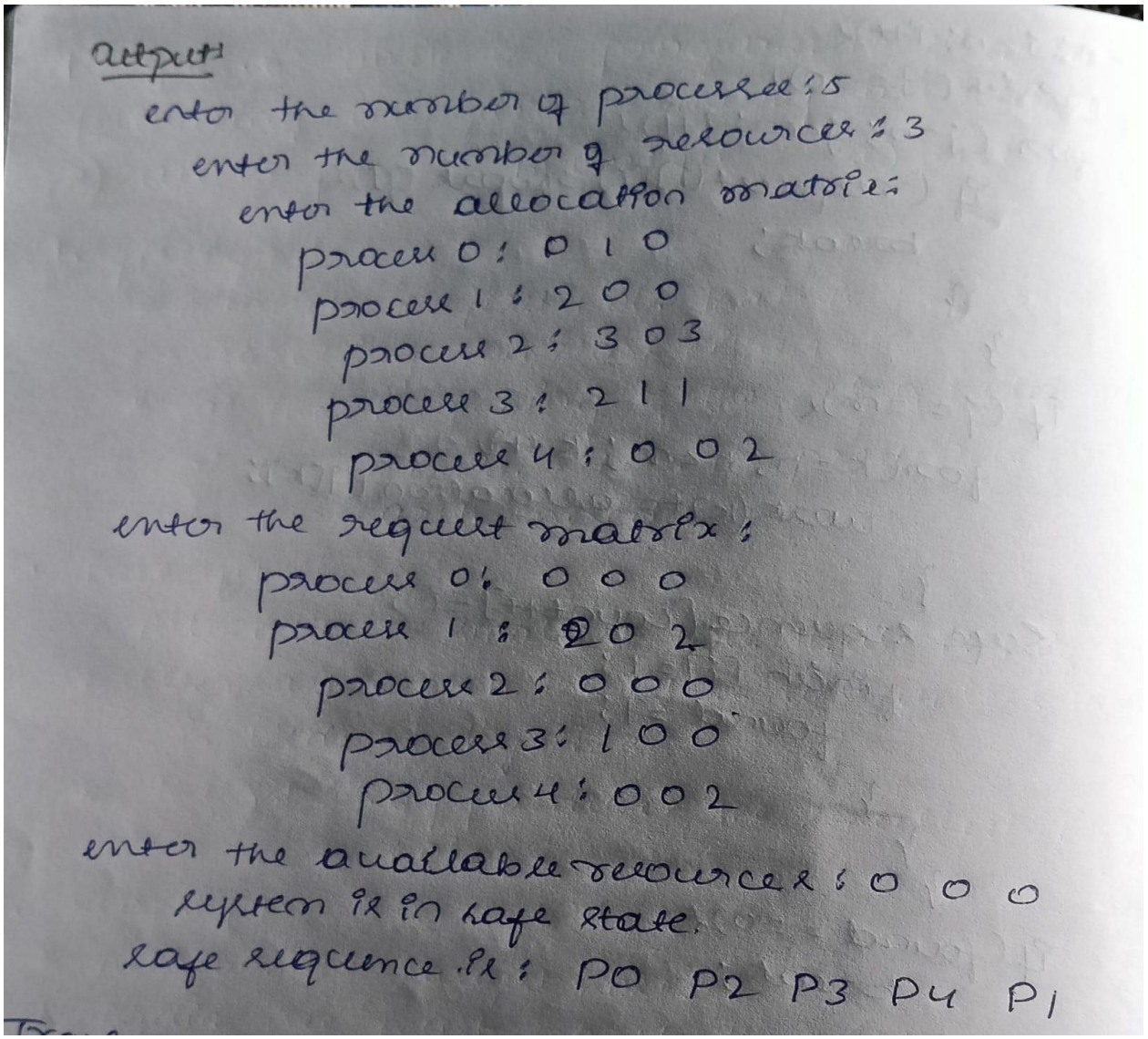
CODE:

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int allocation[n][m], request[n][m];
   int available[m];
```

```
printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++)
   for(j = 0; j < m; j++)
      scanf("%d", &allocation[i][j]);

printf("Enter Request Matrix:\n");
for(i = 0; i < n; i++)
   for(j = 0; j < m; j++)
      scanf("%d", &request[i][j]);

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
   scanf("%d", &available[i]);

int finish[n], work[m], safeSeq[n];

for(i = 0; i < n; i++)
   finish[i] = 0;

for(i = 0; i < m; i++)
   work[i] = available[i];

int count = 0;

while(count < n)
{
   int found = 0;

   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         for(j = 0; j < m; j++)
         {
            if(request[i][j] > work[j])
               break;
         }

         if(j == m)
         {
            for(k = 0; k < m; k++)
               work[k] += allocation[i][k];
```

```
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

if(count == n)
{
    printf("\nNo Deadlock Detected\n");
    printf("Safe Sequence: ");

    for(i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);

    printf("\n");
}
else
{
    printf("\nDeadlock Detected.\nProcesses in Deadlock: ");

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
            printf("P%d ", i);
    }
    printf("\n");
}

return 0;
}
```

OUTPUT:



Output:

enter the number of processes: 5
enter the number of resources: 3
enter the allocation matrix:
process 0: 0 1 0
process 1: 2 0 0
process 2: 3 0 3
process 3: 2 1 1
process 4: 0 0 2
enter the request matrix:
process 0: 0 0 0
process 1: 2 0 2
process 2: 0 0 0
process 3: 1 0 0
process 4: 0 0 2
enter the available resources: 0 0 0
system is in safe state.
safe sequence is: P0 P2 P3 P4 P1

PROGRAM-6:

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit b) Best-fit c) First-fi

CODE:

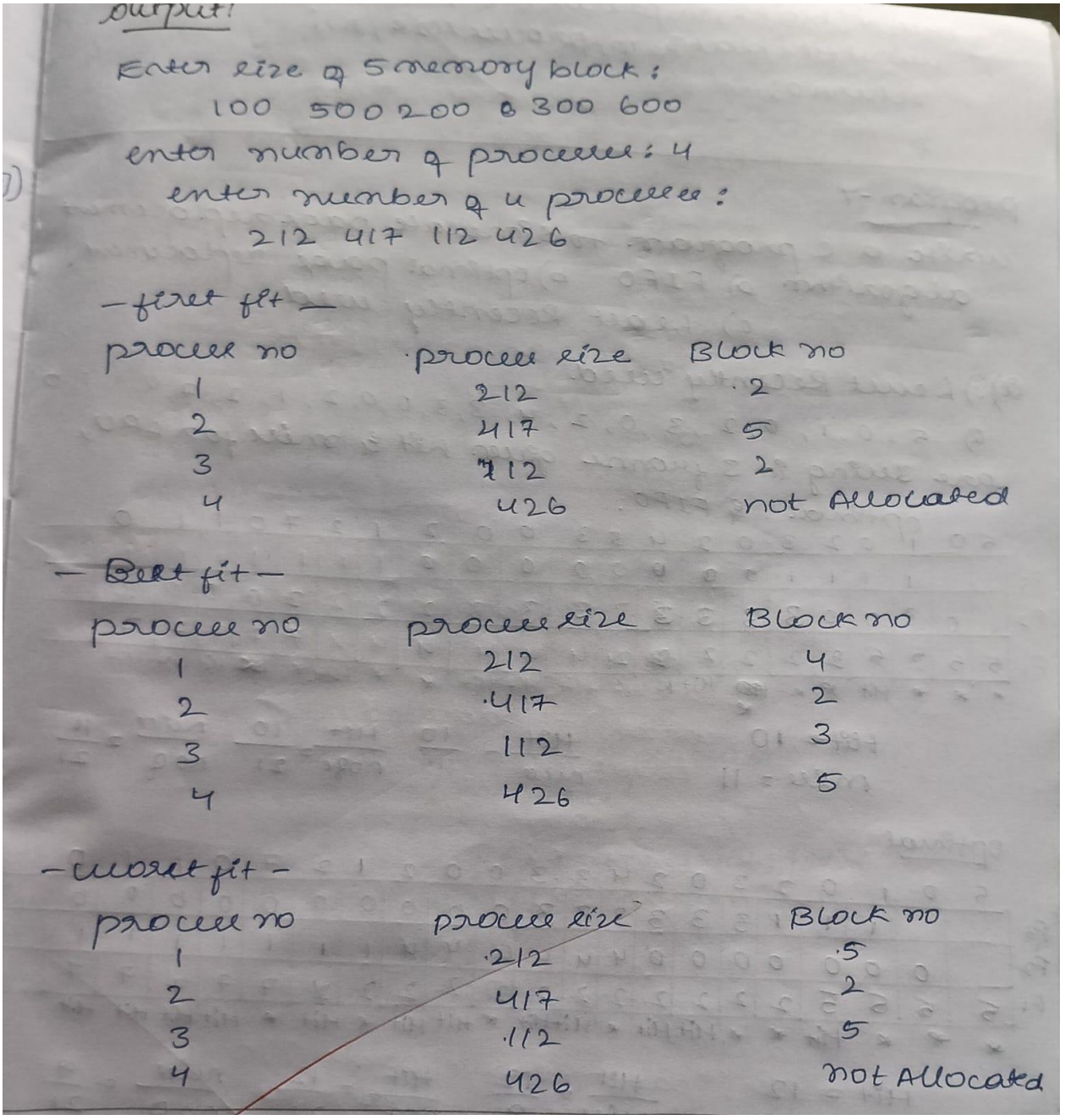
```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n)
{
   int allocation[n];

   for(int i = 0; i < n; i++)
      allocation[i] = -1;

   for(int i = 0; i < n; i++)
   { for(int j = 0; j < m;
      j++)
      { if(blockSize[j] >=
         processSize[i])
         { allocation[i] =

            j;

            blockSize[j]

            = 0;

            break;
         }
      }
   }

   printf("\n--- First Fit ---\n");
   printf("Process No.\tProcess Size\tBlock
   No.\n");

   for(int i = 0; i < n; i++)
   { printf("%d\t\t%d\t\t", i + 1,
      processSize[i]);

      if(allocation[i] != -1)
         printf("%d\n", allocation[i] +
         1);
```

```
        else printf("Not
            Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int
n) {
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    { int bestIdx = -1;
        for(int j = 0; j < m;
        j++)
        { if(blockSize[j] >=
            processSize[i])
            { if(bestIdx == -1 || blockSize[j] <
                blockSize[bestIdx])
                { bestIdx =
                    j;
                }
            }
        }

        if(bestIdx != -1)
        { allocation[i] =
            bestIdx;
            blockSize[bestIdx]
            = 0;
        }
    }

    printf("\n--- Best Fit ---\n"); printf("Process
    No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    { printf("%d\t\t%d\t\t", i + 1,
        processSize[i]); if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
```

```
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for(int i = 0; i < n;
    i++) allocation[i] =
    -1;

    for(int i = 0; i < n; i++)
    {
        int worstIdx = -1;
        for(int j = 0; j < m;
        j++)
        { if(blockSize[j] >=
            processSize[i])
            { if(worstIdx == -1 || blockSize[j] >
                blockSize[worstIdx])
                { worstIdx =
                    j;
                }
            }
        }

        if(worstIdx != -1)
        { allocation[i] =
            worstIdx;
            blockSize[worstIdx]
            = 0;
        }
    }

    printf("\n--- Worst Fit ---\n"); printf("Process
    No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    { printf("%d\t\t%d\t\t", i + 1,
        processSize[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
```

```
            printf("Not Allocated\n");
    }
}

int main()
{ int m,
    n;

    printf("Enter number of memory blocks: ");

    scanf("%d", &m); int blockSize[m], block1[m],

    block2[m], block3[m]; printf("Enter sizes of %d

    memory blocks:\n", m);

    for(int i = 0; i < m; i++)
    { scanf("%d",
        &blockSize[i]); block1[i]
        = blockSize[i]; block2[i]
        = blockSize[i]; block3[i]
        = blockSize[i]; }

    printf("\nEnter number of processes: ");

    scanf("%d", &n); int processSize[n];

    printf("Enter sizes of %d processes:\n",

    n);

    for(int i = 0; i < n; i++)
    { scanf("%d",
        &processSize[i]);
    }

    firstFit(block1, m, processSize, n);
    bestFit(block2, m, processSize, n);
    worstFit(block3, m, processSize, n);

    return 0;
}
```

OUTPUT:



output:

Enter size of 5 memory block:
100 500 200 300 600
enter number of process: 4
enter number of 4 process:
212 417 112 426

-first fit-

process no	process size	Block no
1	212	2
2	417	5
3	112	2
4	426	not Allocated

- Best fit -

process no	process size	Block no
1	212	4
2	417	2
3	112	3
4	426	5

-worst fit -

process no	process size	Block no
1	212	5
2	417	2
3	112	5
4	426	not Allocated

PROGRAM-7:

Write a C program to simulate page replacement algorithms

a) FIFO b) LRU c) Optimal

Code:

```
#include <stdio.h>

#define MAX 100

int isPresent(int frames[], int num_frames, int
page)
{ for (int i = 0; i < num_frames;
   i++)
   { if (frames[i] ==
      page)
         return
   1; }

   return
0; }


void printFrames(int frames[], int num_frames)
{
   printf("[");
   for (int i = 0; i < num_frames; i++)
   { if (frames[i] != -
      1)
         printf("%d ", frames[i]);
   }

   printf("]\n"
); }


void FIFO(int pages[], int n, int num_frames)
{ int
   frames[MAX];
   int page_faults
   = 0; int next =
   0;
```

```
   for (int i = 0; i < num_frames; i++)
      frames[i] = -1; printf("\n--- FIFO Page

   Replacement ---\n"); for (int i = 0; i < n;

   i++)

   { int page =
      pages[i];

      if (!isPresent(frames, num_frames, page))
      { frames[next] = page; next =
      (next + 1) % num_frames;
      page_faults++; }

      printf("Page %d -> ", page);
      printFrames(frames, num_frames);
   }

   printf("Total Page Faults (FIFO): %d\n", page_faults);
}


void LRU(int pages[], int n, int num_frames)
{ int
   frames[MAX];
   int
   lastUsed[MAX];
   int page_faults =
   0;

   for (int i = 0; i < num_frames; i++)
   { frames[i] = -1; lastUsed[i] = -1; }
   printf("\n--- LRU Page Replacement ---
   \n");

   for (int i = 0; i < n; i++)
   { int page =
      pages[i]; int
      found = 0;

      for (int j = 0; j < num_frames; j++)
```

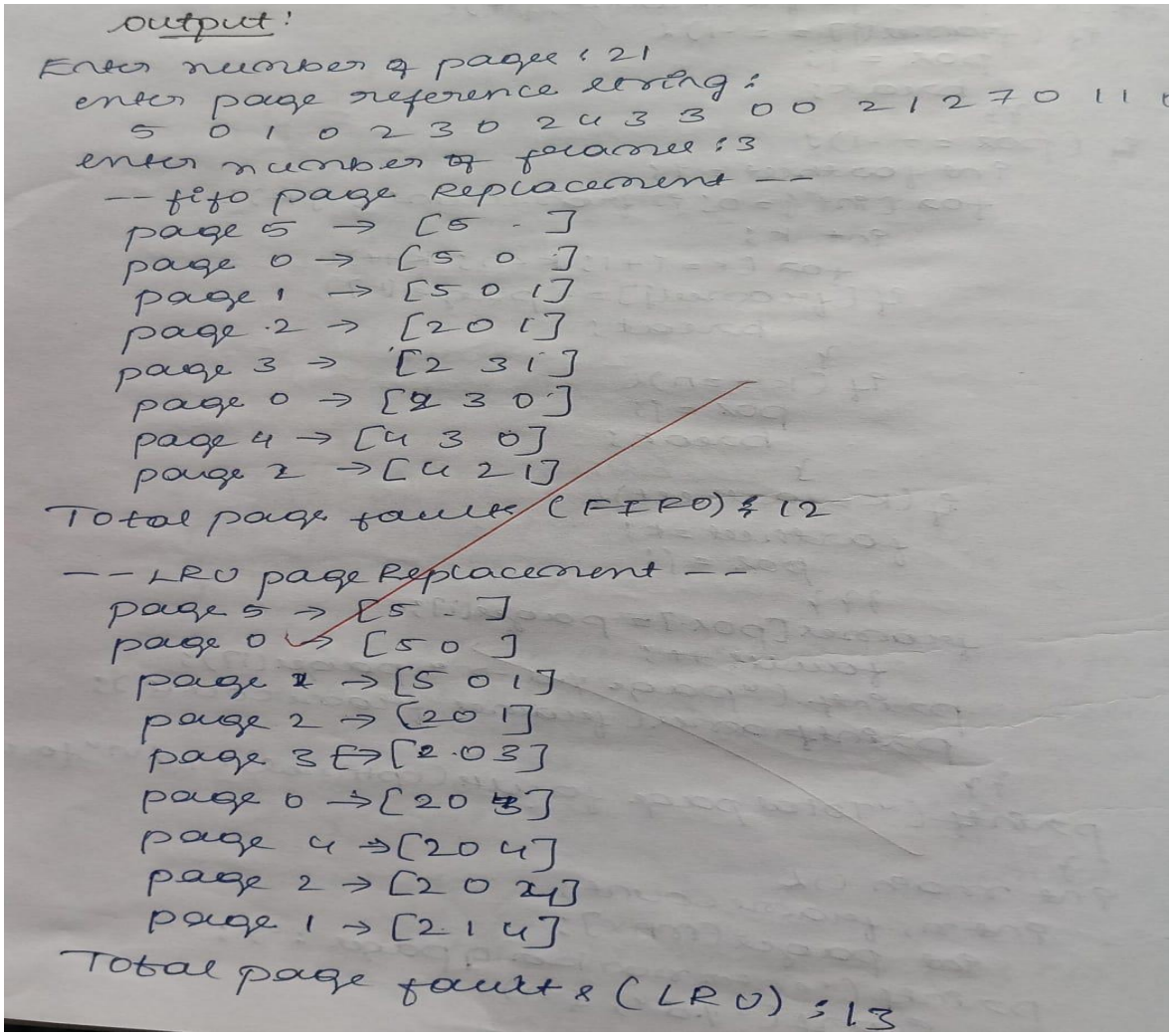
```
    { if (frames[j] ==
       page)
       { found =
          1;
          lastUsed[j] = i;
          break;
       }
    }

    if (!found) {

    int pos = -1;

       for (int j = 0; j < num_frames; j++)
       { if (frames[j] == -
          1)
          { pos =
             j;
             brea
             k;
          }
       }

       if (pos == -1)
       { int lruIndex =
          0;

          for (int j = 1; j < num_frames; j++)
          { if (lastUsed[j] <
             lastUsed[lruIndex])
                lruIndex =
          j; }

          pos =
       lruIndex; }

       frames[pos] =
    page; lastUsed[pos]
    = i; page_faults++;
    }

    printf("Page %d -> ", page);
    printFrames(frames, num_frames);
}
```

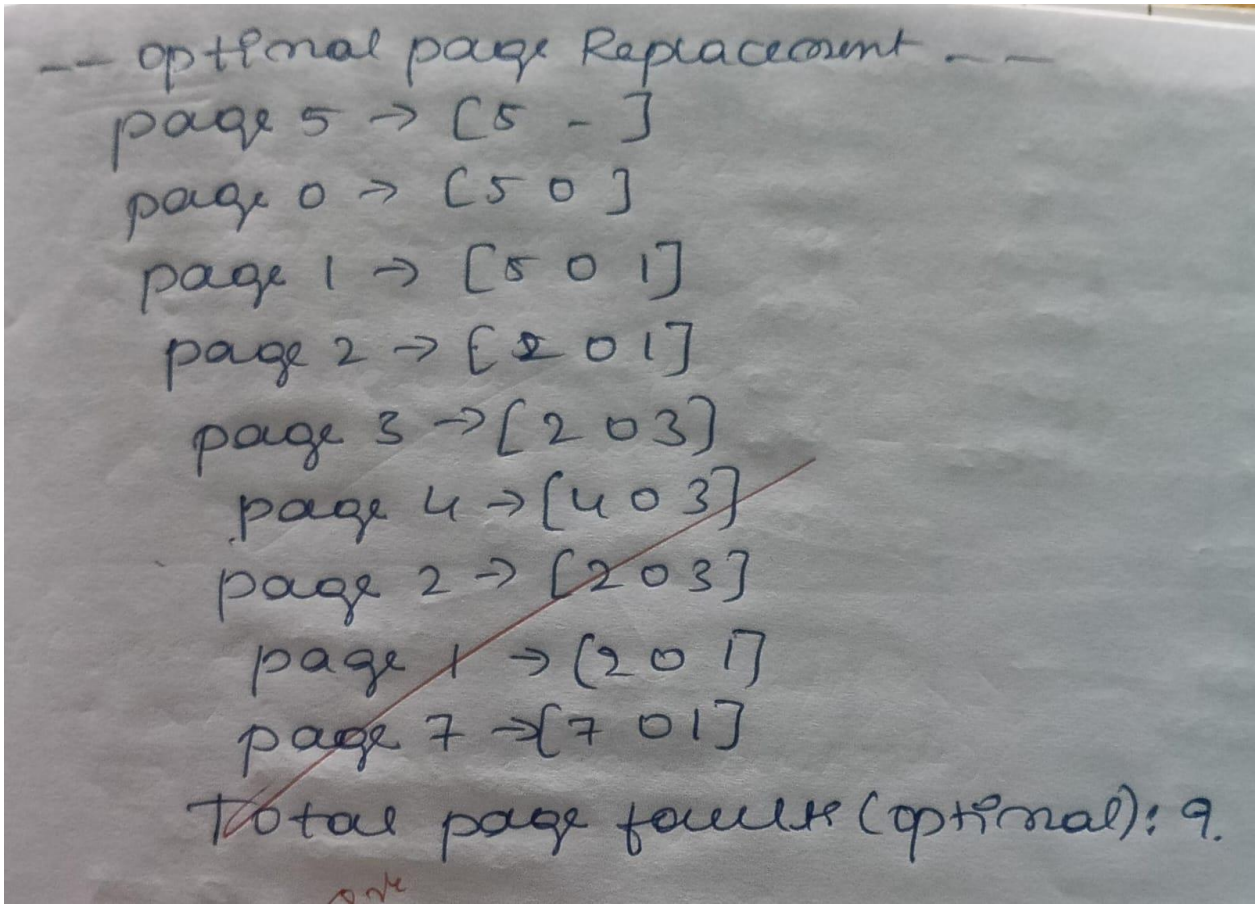
```
                printf("Total Page Faults (LRU): %d\n",
                                                page_faults);
}


void Optimal(int pages[], int n, int num_frames)
{ int
   frames[MAX];
   int page_faults
   = 0;
   // Initialize frames for (int i = 0;
   i < num_frames; i++)
      frames[i] = -1;


   printf("\n--- Optimal Page Replacement ---
   \n");

   for (int i = 0; i < n;

   i++) { int page =

   pages[i];

     if (!isPresent(frames, num_frames, page))
       { int pos =
          -1;

          for (int j = 0; j < num_frames;
          j++) {

             if (frames[j] == -1)
             { pos =
                j;
                brea
                k;
             }
          }

          if (pos == -
          1) {

             int farthest = i; int
             replaceIndex = -1;
```

```
            for (int j = 0; j < num_frames; j++)
            { int
               k;

               for (k = i + 1; k < n; k++)
               { if (frames[j] ==
                  pages[k])
                     brea
               k; }

               if (k == n)
               { replaceIndex =
                  j; break;
               }

               if (k > farthest)
               { farthest = k;
                  replaceIndex
                  = j;
               }
            }

            pos = replaceIndex;
         }

         frames[pos] =
      page;
      page_faults++; }

      printf("Page %d -> ", page);
      printFrames(frames, num_frames);
   }

   printf("Total Page Faults (Optimal): %d\n", page_faults);
}


int main()
{ int pages[MAX], n,
   num_frames;

   printf("Enter number of pages: ");
   scanf("%d", &n);
```

```
    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++)
    { scanf("%d",
       &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &num_frames);

    FIFO(pages, n, num_frames);
    LRU(pages, n, num_frames);
    Optimal(pages, n, num_frames);

    return 0;
}
```

OUTPUT:




-- optimal page Replacement --
page 5 → [5 -]
page 0 → [5 0]
page 1 → [5 0 1]
page 2 → [2 0 1]
page 3 → [2 0 3]
page 4 → [4 0 3]
page 2 → [2 0 3]
page 1 → [2 0 1]
page 7 → [7 0 1]
Total page fault (optimal): 9.